

Evaluating ChatGPT-5 for Misuse Case Diagram Generation: An Empirical Evaluation

Alia Alzarooni¹, Yasser Khan², Hassan Alsayegh¹, Mohamed El-Attar¹, Rima Grati¹

¹College of Technological Innovation, Zayed University, Abu Dhabi, United Arab Emirates

²Department of Electrical Engineering and Computer Science, Florida Atlantic University, Florida, USA
{M80009248, M80009296, Mohamed.El-Attar, Rima.Grati}@zu.ac.ae, khany2026@fau.edu

Keywords: Misuse Case Diagrams, Security Requirements Engineering, Large Language Models, ChatGPT-5, UML, Threat Modeling, Empirical Evaluation

Abstract: Misuse case diagrams are a widely adopted technique in security requirements engineering, enabling analysts to model adversarial threats and derive countermeasures early in the software development lifecycle. However, manual construction of these diagrams is prone to incompleteness and subjectivity, requiring significant security expertise. Large language models (LLMs) such as ChatGPT present a promising opportunity to automate this process, yet their effectiveness for generating structured security modeling artifacts remains largely unexplored. This paper presents an exploratory study evaluating ChatGPT-5's ability to generate misuse case diagrams directly from textual security requirements, using 12 case studies of varying complexity spanning small, medium, and large requirement sets. The diagrams produced by ChatGPT-5 were evaluated against manually constructed ground-truth diagrams, and our results indicate that ChatGPT-5 performs well overall, demonstrating a strong capability to identify key actors, threats, and adversarial relationships from natural language input.

1 INTRODUCTION

Modern software systems increasingly require security to be integrated during the early phases of design rather than introduced as a reactive mechanism after implementation. As cyber-attacks grow in sophistication, security engineering practices have shifted toward requirements-level approaches that enable analysts to anticipate vulnerabilities and evaluate system misuse before deployment (Howard and Lipner, 2006; Shostack, 2014), aligning with secure-by-design principles that emphasize modeling threats at the system-design stage to prevent security flaws from propagating into later development phases (El-Attar, 2012; OWASP, 2021).

Among the techniques developed to support this goal, misuse case diagrams have emerged as a widely

adopted extension of UML use case diagrams that explicitly represent malicious actors, adversarial interactions, and mitigation mechanisms alongside legitimate system functionality (Sindre and Opdahl, 2005; El-Attar, 2012; El-Attar, 2014). However, their manual construction is prone to incompleteness and subjectivity, demands significant security expertise, and is difficult to scale as system complexity grows.

The rapid advancement of LLMs has opened new possibilities for automating such tasks. This study employs ChatGPT-5 in Auto mode, which represents a substantial advancement over ChatGPT-4 in reasoning depth, instruction-following precision, and structural consistency on complex generation tasks (Cámara et al., 2023; Borji, 2023), making it well suited for misuse case diagram generation where security semantics, adversarial relationships, and

^a <https://orcid.org/0009-0002-1359-5130>

^b <https://orcid.org/0009-0008-9666-0615>

^c <https://orcid.org/0009-0002-1070-7163>

^d <https://orcid.org/0000-0002-6709-8212>

^e <https://orcid.org/0009-0002-6995-465X>

notational correctness must be managed simultaneously. Hereafter the model is referred to simply as ChatGPT. Despite growing interest in applying LLMs to software engineering tasks (Jiang et al., 2024; Sridhara and Mazumdar, 2023), limited research has evaluated whether such models can reliably generate accurate misuse case diagrams from informal natural language descriptions (Berry et al., 2003; Kamalrudin, 2013), a gap this paper directly addresses.

This paper presents an exploratory study evaluating ChatGPT's ability to generate cybersecurity misuse case diagrams from natural language security requirements across 12 case studies of varying complexity. Generated diagrams are evaluated against expert-constructed ground-truth baselines, and the study identifies recurring limitations with implications for AI-assisted security modeling workflows. To guide this investigation, the study addresses the following research questions:

RQ1 Syntactic Correctness: To what extent does ChatGPT generate syntactically correct misuse case diagrams that conform to the established misuse case notation, including the correct use of actors, misuse cases, and relationships?

RQ2 Semantic Correctness: How accurately does ChatGPT capture the intended security semantics of the input requirements, including the correct identification of threat actors, misuse scenarios, and mitigation mechanisms?

RQ3 Visual/Notational Quality: To what extent do ChatGPT-generated misuse case diagrams exhibit visual clarity and notational coherence that supports cognitive understanding of adversarial threat scenarios?

RQ4 Scalability: How does the complexity and scale of the input security requirements affect the syntactic and semantic quality of misuse case diagrams generated by ChatGPT?

RQ5 Domain Sensitivity: How sensitive is ChatGPT to the domain of the input security requirements?

RQ6 Cross-Session Context Retention: To what extent does cross-session context retention in ChatGPT affect the efficacy of the LLM in generating correct diagrams with fewer corrective prompts?

The remainder of this paper is organized as follows. Section 2 provides background on misuse case modeling and related work. Section 3 describes the research methodology. Section 4 presents and discusses the results. Section 5 concludes the paper and outlines directions for future work

2 BACKGROUND AND RELATED WORK

2.1 Misuse Case Modeling

Unified Misuse case modeling is a security-oriented extension of the Unified Modeling Language (UML) use case notation, originally proposed by Sindre and Opdahl (2005) as a systematic technique for eliciting and representing security requirements during early system design. While traditional UML use case diagrams capture the intended interactions between legitimate actors and system functionality, they do not account for adversarial behavior or the intentional exploitation of system services. Misuse case modeling addresses this limitation by introducing additional modeling elements that explicitly represent malicious actors, adversarial actions, and defensive countermeasures alongside normal system operations.

Figure 1 illustrates a simple misuse case diagram for an online purchasing system and demonstrates the core elements of the notation. Legitimate actors are represented using the standard UML stick figure symbol with a white head, as shown by the "Customer". Misusers, representing malicious or hostile entities, are distinguished visually as represented by the "Crook". Use cases representing legitimate system functionality are depicted as white ellipses, such as Submit Purchasing Order and Encrypt Data. Misuse cases representing adversarial actions are depicted as filled black ellipses, such as Steal Credit Card Information and Intrusion Detected.

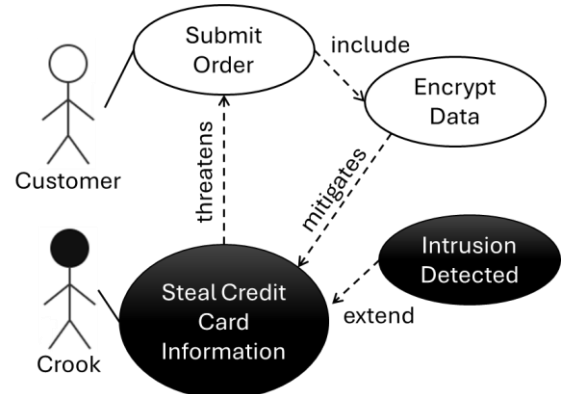


Figure 1 Example misuse case diagram

The relationships between these elements capture the adversarial dynamics of the system. The Crook is associated with the Steal Credit Card Information misuse case, which threatens the Submit Purchasing Order use case, illustrated by a directed arrow

labelled "threatens" pointing from the misuse case to the use case it targets. The Submit Purchasing Order use case includes the Encrypt Data use case as a mandatory security dependency, represented by a dashed arrow labelled «include». The Encrypt Data use case mitigates the Steal Credit Card Information misuse case, shown by a directed line labelled "mitigates" connecting the defensive mechanism to the threat it counters. Finally, the Intrusion Detected misuse case extends the Steal Credit Card Information misuse case via an «extend» relationship, indicating a conditional adversarial behavior triggered under specific circumstances.

[Insert Figure 1: Misuse Case Diagram for an Online Purchasing System]

By making adversarial intent explicit within the requirements model, misuse case diagrams support secure-by-design development practices and enable engineers to identify vulnerabilities and define countermeasures before implementation begins (El-Attar, 2012; Howard and Lipner, 2006). However, the manual construction of these diagrams demands significant security expertise and is prone to incompleteness and subjectivity, motivating the investigation of automated approaches such as those enabled by large language models.

2.2 LLMs for Security Modeling and Threat Analysis

The application of large language models to software engineering tasks has grown substantially in recent years, with studies demonstrating their utility for requirements classification, test case generation, code completion, and UML diagram generation. Cámara et al. (2023) conducted a systematic assessment of ChatGPT's ability to perform general UML modeling tasks, evaluating syntactic correctness, semantic alignment, scalability, and sensitivity to prompt variation. Their findings revealed that while ChatGPT could produce structurally recognizable UML diagrams, performance degraded significantly as model complexity increased, and semantic correctness was inconsistent across prompt variations. Importantly, their study focused exclusively on general-purpose UML modeling and did not examine security-specific artifacts such as misuse case diagrams, leaving a significant gap in understanding how LLMs perform in adversarial modeling contexts.

Research specifically targeting LLMs in cybersecurity and threat modeling contexts has begun to emerge but remains limited in scope. AbdulGhaffar and Matrawy (2025) evaluated five LLMs on their

ability to classify security threats within the STRIDE framework, finding that while models performed reasonably well on explicit threat descriptions, they were prone to misclassification when input descriptions were ambiguous or lacked domain-specific keywords. This highlights a key challenge in security contexts: the effectiveness of LLM-assisted modeling is closely tied to the clarity and precision of the input, and performance may degrade when analysts express requirements informally. Saha et al. (2025) introduced ThreatLens, a multi-agent LLM framework for automating threat modeling in hardware systems using retrieval-augmented generation and iterative self-refinement. Although their approach improved threat coverage, it relied on well-structured architecture specifications and did not explore how models interpret informal or incomplete natural language input, nor did it evaluate diagrammatic outputs. Similarly, Wu et al. (2024) applied LLMs to automate threat modeling for banking systems, demonstrating promising results but again relying on structured input formats and producing textual rather than visual outputs.

Research on human-AI interaction further underscores the challenge of intent interpretation in AI-assisted modeling. Khojah et al. (2024) demonstrated that the effectiveness of LLMs in software engineering tasks is closely tied to the clarity and structure of user input, and that early-stage descriptions expressed in informal or fragmented language significantly reduce output quality. In cybersecurity contexts this challenge is amplified, as analysts often describe vulnerabilities and attacker behavior using incomplete statements that implicitly reference relationships between legitimate system functions and malicious actions. Transforming such descriptions into structured visual artifacts requires the model to infer semantic relationships that are not explicitly stated in the input, a capability that has not been systematically evaluated for misuse case diagram generation.

Taken together, the existing literature reveals a clear gap: while LLMs have been applied to general UML modeling and to textual threat analysis tasks, no study has systematically evaluated their ability to generate visual misuse case diagrams from natural language security requirements across varying levels of complexity and application domain. This paper directly addresses that gap by providing the first structured empirical evaluation of ChatGPT's performance on this specific task.

3 EXPLORATORY RESEARCH METHODOLOGY DESIGN

This section describes the methodology adopted to evaluate ChatGPT's ability to generate cybersecurity misuse case diagrams from natural language security requirements. The study follows a structured, evidence-driven evaluation design grounded in established empirical software engineering research practices (Wohlin et al., 2012). This methodological orientation is consistent with prior work evaluating LLMs for modeling and diagram generation tasks, most notably Cámara et al. (2023), who adopted a similarly structured empirical approach to assess ChatGPT's performance on general UML modeling tasks by comparing model outputs against expert-defined reference models across varying levels of complexity. Our study extends this evaluation paradigm into the domain of security-specific modeling, applying a comparable evidence-driven framework to misuse case diagram generation.

3.1 Case Studies and Ground-Truth Diagrams

The evaluation is conducted using 12 case studies of varying complexity, categorized as small (3 cases), medium (7 cases), and large (2 cases). Case study size is defined by the number of notational constructs that compose the corresponding misuse case diagram,

with small cases containing fewer than 15 constructs, medium cases containing between 16 and 40 constructs, and large cases containing more than 41 constructs. The case studies are drawn from a mix of real-world and synthetic sources, spanning diverse application domains including cyber-physical systems, administrative environments, and financial systems, enabling the evaluation of both scalability and domain sensitivity in line with RQ4 and RQ5. The use of case studies of varying size and domain is consistent with the evaluation designs adopted by Cámara et al. (2023) and Aruna and Varun (2025), both of whom similarly varied the complexity of modeling tasks to assess how LLM performance scales with input size and domain specificity.

For each case study, an expert-designed ground-truth misuse case diagram was manually constructed prior to any interaction with ChatGPT, capturing the correct set of actors, misusers, use cases, misuse cases, mitigation use cases, and all associated relationships including «threatens», «mitigates», «include», and «extend». The use of expert reference diagrams as an evaluation baseline is a well-established practice in empirical studies of AI-assisted modeling, adopted by Mbaka and Tuma (2024) for LLM-generated threat diagrams. In our study, the ground-truth diagrams were constructed independently by two security engineering experts with extensive experience in misuse case modeling, and disagreements were resolved through discussion until consensus was reached.

Table 1 Descriptive statistics of the 12 intended misuse cases diagrams

Model	Size	Actors	Misusers	Use cases	Misuse cases	Include	Extends	Threatens	Mitigates	Total
Food Delivery	Small	1	1	3	1	1	0	1	1	9
Cloud Storage	Small	1	1	4	1	3	0	1	1	12
Library	Small	1	1	4	2	2	1	1	1	13
Rideshare	Medium	2	2	7	3	3	1	3	3	24
Smart Home	Medium	1	2	7	3	3	1	4	3	24
ATM	Medium	2	2	6	3	6	0	3	3	25
Online Exam	Medium	2	2	7	3	3	1	3	4	25
Online Store	Medium	1	2	7	4	2	0	5	4	25
E-Voting	Medium	2	2	8	3	4	1	3	3	26
Healthcare	Medium	3	2	8	4	4	0	4	5	30
RFID	Large	3	1	12	7	17	0	6	8	54
Swiss Bank	Large	3	2	12	6	19	0	4	8	54

Descriptive statistics characterizing the notational constructs of each ground-truth diagram, including the number of actors, misusers, use cases, misuse cases, mitigation use cases, and relationships, are presented in Table 1 to provide a structural profile of each case study and to contextualize the complexity of the modeling task presented to the LLM. A brief description of each case study is provided below:

Case Study 1: Food Delivery System (Small)

The Food Delivery System is a mobile-based platform that enables customers to place food orders and track the status of their deliveries in real time. The system supports the end-to-end order fulfillment process, from order submission through to delivery confirmation, and provides customers with live visibility into the progress of their orders. Delivery verification is handled through a PIN code mechanism to confirm successful receipt of the order at the customer's location.

Case Study 2: Cloud Storage System (Small)

The Cloud Storage System is a file management platform that allows registered users to upload and download files to and from a centralized cloud repository. The system supports secure file transfer operations and manages user access through an authentication mechanism that must be satisfied before any file transaction can take place. File transfer operations are protected through encryption to ensure the confidentiality and integrity of data in transit.

Case Study 3: Library Management System (Small)

The Library Management System is an institutional platform that manages the borrowing and return of books by registered library members. The system supports core lending operations including member verification prior to book borrowing and the processing of book returns. Membership identity is confirmed through the scanning of a member photo ID, ensuring that only authorized individuals are permitted to borrow materials from the library collection.

Case Study 4: Ride Sharing System (Medium)

The Ride Sharing System is a transportation platform that connects passengers with drivers to facilitate on-demand ride bookings. The system supports key operational functions including ride requests, real-time location tracking, and payment processing. Passengers are able to split fares for shared journeys, and payment pre-authorization is required before a ride is confirmed. Driver identity is verified as part of

the ride request process, and passengers are provided with the option to share ride details with a trusted contact for safety purposes.

Case Study 5: Smart Home System (Medium)

The Smart Home System is an internet-connected home automation platform that enables homeowners to remotely manage and monitor various smart devices within their residence. The system supports a range of functions including remote door locking, live security camera viewing, and real-time monitoring of device communication. The platform integrates intrusion alert notifications that can be extended to notify emergency contacts in the event of a detected security incident, and biometric verification is required to authorize remote access to physical entry points.

Case Study 6: ATM System (Medium)

The Bank ATM System is a self-service banking terminal that enables customers and bank staff to perform financial transactions including cash withdrawals and fund transfers. The system requires customer authentication prior to any transaction and incorporates card data encryption to protect sensitive financial information during processing. Transaction alerts are generated automatically to notify account holders of activity, providing an additional layer of operational transparency and supporting the bank's fraud monitoring responsibilities.

Case Study 7: Online Exam System (Medium)

The Online Exam System is an academic assessment platform that supports the creation and administration of examinations for students by instructors. Instructors are responsible for creating exams, which include configuring access time windows and randomizing question order to maintain assessment integrity. Students interact with the system to take exams, submit their answers, and may request time extensions under applicable circumstances. The platform incorporates webcam-based proctoring during active exam sessions to supervise student conduct and uphold the validity of the assessment process.

Case Study 8: Online Store System (Medium)

The Online Store System is an e-commerce platform that allows customers to browse a product catalogue, place orders, and make payments for purchased items. The system manages the full customer purchase journey from product discovery through to payment completion and incorporates user authentication as part of the ordering process. Communications

between the customer and the system are encrypted using SSL/TLS protocols, and payment verification mechanisms are applied to ensure the integrity of financial transactions processed through the platform.

Case Study 9: E-Voting System (Medium)

The E-Voting System is a digital electoral platform that facilitates the casting and tallying of votes in an election managed by election officials. Registered voters interact with the system to cast their votes, while election officials are responsible for tallying results, publishing outcomes, and processing recount requests where applicable. The system maintains an audit trail to support the transparency and accountability of the electoral process, and voter eligibility verification is performed to ensure that only authorized individuals are permitted to participate in the election.

Case Study 10: Healthcare Information System (Medium)

The Healthcare Information System is a clinical records platform that supports the management of patient health information across a multi-role healthcare environment involving nurses, doctors, and patients. The system enables authorized clinical staff to update medical histories and view patient records, while patients are able to request appointments through the platform. Medication prescribing is supported as a clinical function within the system, and patient identity verification is required as part of the appointment request workflow to ensure accurate record association and care delivery.

Case Study 11: RFID System (Large)

The RFID System is an asset authentication and tracking platform used by supervisors and product managers to authenticate products and manage supply chain integrity through radio-frequency identification technology. The system supports product authentication operations that incorporate location-based authentication and mechanisms to prevent and detect tag removal. Product managers are responsible for tag authentication activities, which include verifying the integrity and authenticity of product history records. The platform interfaces with both RF communication channels and internal IT infrastructure to support end-to-end product traceability across the supply chain.

Case Study 12: Swiss Bank System (Large)

The Swiss Bank System is a private banking platform that supports the full spectrum of client relationship

and financial management activities across multiple organizational roles including the Director, Customer Representatives in the front office, and Account Managers in the middle office. The system facilitates core banking operations such as maintaining business relationships, complying with country regulations, enrolling new clients, taking client orders, and providing returns on investment to clients. Client identity verification, transaction monitoring, and the rejection of suspicious clients or transfers above defined thresholds are integral operational functions, and the system is designed to alert relevant authorities in cases of suspected financial irregularity.

3.2 Data Collection and Prompting Protocol

The data collection procedure is based on an iterative prompting protocol designed to systematically evaluate ChatGPT's ability to produce misuse case diagrams that are structurally, semantically, and visually aligned with the corresponding ground-truth diagram. All interactions with ChatGPT were conducted in isolated sessions to prevent cross-prompt contamination and ensure that output variations are attributable solely to differences in input requirements and prompting strategy. This practice of using isolated sessions to control for prompt contamination is consistent with the experimental controls applied by Cámara et al. (2023) and Sridhara and Mazumdar (2023), who similarly isolated individual LLM interactions to ensure the validity of comparative evaluations across tasks.

Each trial run followed a consistent three-phase process. In the first phase, the natural language security requirements for a given case study were submitted to ChatGPT as an initial prompt, with no additional guidance or domain context beyond the requirements text itself. This simulates a realistic scenario in which an analyst provides an informal description of system security needs and requests a corresponding misuse case diagram. Providing only the raw requirements as input, without additional scaffolding or domain priming, reflects the evaluation approach taken by Cámara et al. (2023) and is consistent with the objective of assessing the model's baseline interpretive capability rather than its performance under optimal prompting conditions.

In the second phase, the generated diagram was compared against the ground-truth diagram and evaluated for syntactic and semantic errors. Syntactic errors include violations of the UML misuse case notation, such as incorrect use of relationship types, missing system boundaries, or incorrect actor

representations. Semantic errors include incorrect identification of actors or misusers, missing or incorrectly mapped misuse cases, erroneous threat relationships, and omitted or incorrectly linked mitigation mechanisms. Where such errors were identified, corrective prompts were issued to ChatGPT to guide the model toward the intended representation. These corrective interactions are referred to as construct prompts, as they address the structural and semantic integrity of the diagram. The use of iterative corrective prompting as a refinement mechanism is consistent with the approach adopted by Saha et al. (2025), whose ThreatLens framework employed iterative self-refinement to progressively improve the quality of LLM-generated threat analyses, and by Cámara et al. (2023), who similarly applied follow-up prompts to assess the model's capacity for correction and refinement. The number of construct prompts required per trial run was recorded as a primary evaluation metric.

In the third phase, once semantic and syntactic correctness had been achieved, the diagram was further evaluated for visual presentation quality. Presentation errors include issues such as overlapping elements, unclear layout, illegible labels, or poor spatial organization that impede the cognitive readability of the diagram regardless of its notational correctness. Corrective interactions targeting these issues are referred to as visual prompts. The number of visual prompts required per trial run was recorded separately from construct prompts to distinguish between errors of semantic substance and errors of presentational form. This separation of semantic and presentational evaluation criteria extends the evaluation framework used by Cámara et al. (2023), who assessed syntactic and semantic correctness but did not explicitly evaluate the visual and cognitive quality of generated diagram outputs, representing a methodological contribution of the present study.

All prompts issued during each trial run, together with the corresponding model outputs, were systematically logged to ensure reproducibility and to support subsequent quantitative and qualitative analysis. All experimental artifacts, prompts and statistical results are available at a companion GitHub repository (Khan and El-Attar, 2026). Each trial run was ultimately classified as either successful, where the final generated diagram was judged to be semantically and syntactically equivalent to the ground-truth diagram, or unsuccessful, where full alignment with the ground truth could not be achieved regardless of the number of prompts issued.

3 Evaluation Metrics

For each trial run, the number of construct prompts required to achieve syntactic and semantic correctness and the number of visual prompts required to achieve acceptable presentation quality were recorded, along with the overall trial run outcome. Error patterns observed across the trial runs were noted and are reported descriptively to characterize the nature and frequency of recurring issues across case studies of varying complexity and domain. Together these measures provide the empirical basis for answering the six research questions presented in Section 1.

4 RUNNING EXAMPLE: LIBRARY MANAGEMENT SYSTEM

To illustrate the misuse case model generation process, we use the Library Management System as a running example, a small system containing 13 constructs that required four prompts in total to arrive at the intended model.

The first prompt provided ChatGPT with seven natural language requirements describing the system. The initial generated misuse case diagram is shown in Fig. 2. The generated model correctly identified the Library Member actor, the Identity Fraudster misuser, and the use cases "Borrow Book", "Return Book", "Verify Membership", and "Scan Photo ID". The misuse cases "Borrow Under False Identity" and "Steal Borrowed Book" were also correctly generated and were visually distinguishable with a dark background. However, three construct level errors were observed. First, the system boundary rectangle was missing, meaning all use cases were rendered without an enclosing system context. Second, an incorrect threatens relationship was generated between "Borrow Under False Identity" and "Verify Membership", which was not implied by the requirements, as the misuse case should only threaten "Borrow Book". Third, an incorrect association was generated between the Identity Fraudster and "Steal Borrowed Book", when the fraudster should only be associated with "Borrow Under False Identity", with "Steal Borrowed Book" being reached through an extend relationship.

The second prompt addressed all three construct level errors by explicitly instructing ChatGPT to enclose all use cases within a system boundary rectangle, remove the incorrect threatens relationship,

and remove the incorrect misuser association. Figure 3 presents the generated misuse case model after the second prompt. The resulting model correctly resolved all three issues. However, a new visual error was introduced where the Library Member actor was incorrectly styled with a dark background, making it appear as a misuser.

The third prompt instructed ChatGPT to restore the Library Member actor to the default styling with a white background and dark label, reserving the dark background exclusively for misusers. This was successfully resolved. However, the misuse case "Steal Borrowed Book" carried an unnecessarily verbose name, "Steal Borrowed Book (Never Return)", which was not implied by the requirements. The fourth and final prompt instructed ChatGPT to shorten the misuse case name to "Steal Borrowed

Book" by removing the parenthetical annotation. This resolved the last remaining issue and produced the intended model. The final generated misuse case model of the Library Management System is shown in Figure 4.

This running example illustrates a pattern observed across all case studies, where ChatGPT is capable of correctly deriving most semantic constructs from natural language requirements but requires iterative correction to address construct-level errors, visual styling issues, and notational deviations. Furthermore, correction prompts can sometimes introduce new errors, as seen when fixing the misuser association caused incorrect styling of a legitimate actor. Figure 5 presents all prompts used in this case study, from the initial requirements-based prompt through to the final correction prompt.

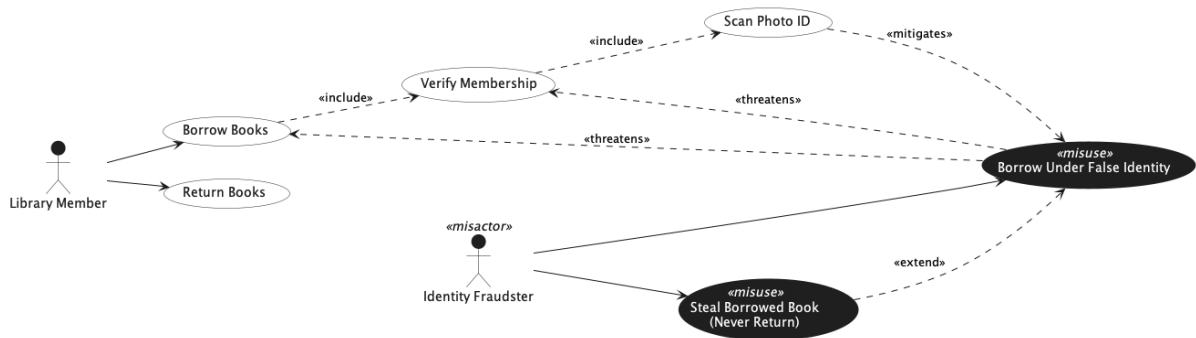


Figure 2 Initial generated misuse case model of the Library Management System.

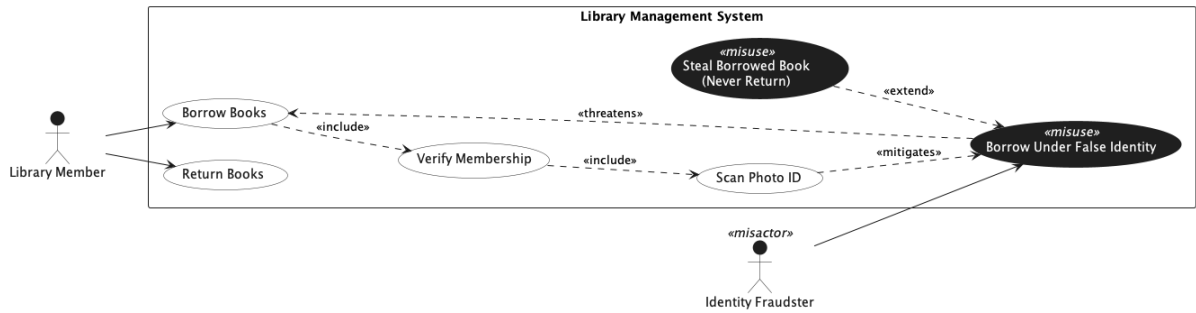


Figure 3 Generated misuse case model after the second prompt.

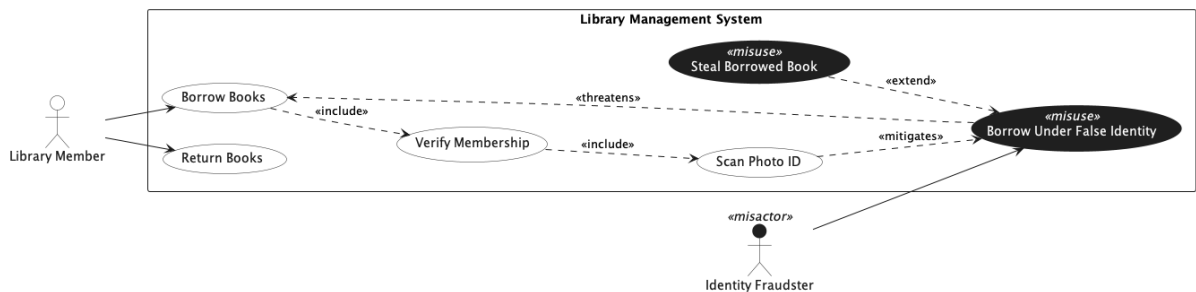


Figure 4 Final generated misuse case model of the Library Management System.

1. Generate a misuse case diagram in PlantUML for a Library Management System based on the following requirements:
 - Req. 1: The system shall allow library members to borrow books.
 - Req. 2: The system shall allow library members to return books.
 - Req. 3: As part of borrowing a book, the system shall always verify the membership of the borrower.
 - Req. 4: As part of verifying membership, the system shall always scan the member's photo ID.
 - Req. 5: An identity fraudster may borrow a book under a false identity, which undermines the book borrowing process.
 - Req. 6: After borrowing a book under a false identity, the fraudster may additionally steal the borrowed book by never returning it.
 - Req. 7: Scanning the member's photo ID shall serve as a countermeasure against borrowing under a false identity.
2. Please make the following corrections. First, enclose all use cases within a system boundary rectangle labelled "Library Management System". Second, remove the <<threatens>> relationship between "Borrow Under False Identity" and "Verify Membership", as the misuse case only threatens "Borrow Book". Third, remove the association between the Identity Fraudster and "Steal Borrowed Book", as the fraudster should only be directly associated with "Borrow Under False Identity".
3. The Library Member actor has been incorrectly styled with a dark background. Please ensure only misusers have a dark background and restore the Library Member actor to the default styling with a white background and dark label.
4. Please shorten the misuse case name to "Steal Borrowed Book" by removing the "(Never Return)" part.

Figure 5 Sequence of prompts for generating the Library Management System misuse case model.

5 RESULTS AND DISCUSSION

This section presents the results of the empirical evaluation conducted across all 12 case studies. Table 2 summarizes the construct and visual prompt counts and overall outcome for each trial run. The case studies are grouped by complexity category, with small, medium, and large systems presented in order of increasing construct count as reported in Table 1.

Table 2 Prompt statistics of the 12 intended misuse cases diagrams

System	Construct Prompts	Visual Prompts	Total
Food Delivery	1	2	3
Cloud Storage	1	0	1
Library	2	2	4
Rideshare	1	1	2
Smart Home	1	1	2
ATM	1	1	2
Online Exam	1	1	2
Online Store	1	1	2
E-Voting	1	3	4
Healthcare	1	2	3
RFID	1	1	2
Swiss Bank	1	0	1

All 12 trial runs were deemed successful. The following subsections discuss the findings in the context of each of the six research questions presented earlier in Section 1.

RQ1: Syntactic Correctness

ChatGPT demonstrated a generally strong ability to conform to UML misuse case notation across all 12 case studies, correctly employing actors, misusers, use cases, misuse cases, system boundaries, and the core relationship types in most cases. However, notational violations were observed in all case studies. In the Healthcare and Ride Sharing case studies, actor-to-use-case associations were rendered as undirected lines rather than the conventional directed arrows. In the Bank ATM case study, include relationships were depicted using solid arrows instead of the required dashed arrows. In three case studies, namely the Online Exam, Online Store, and Smart Home systems, actors and misusers were incorrectly placed inside the system boundary rectangle, violating a fundamental convention of UML use case notation. These findings suggest that while ChatGPT can produce syntactically well-formed misuse case diagrams in most instances, it does not consistently enforce all notational constraints without corrective prompting.

RQ2: Semantic Correctness

ChatGPT demonstrated a strong capability to generate semantically aligned misuse case diagrams from natural language security requirements,

correctly identifying actors, misusers, use cases, misuse cases, and their relationships in the majority of trial runs. Semantic constructs were almost always correctly derived from the initial requirements-based prompt, indicating that the model is effective at interpreting security intent expressed in natural language. However, semantic errors were observed in some cases. ChatGPT introduced relationships not implied by the requirements, such as an unexpected «threatens» relationship and an incorrect misuser association in the Library Management System case study. The model also exhibited a tendency to elaborate on construct names beyond what the requirements specified, adding unnecessary detail and parenthetical annotations. For example, a misuse case was labelled "Steal Borrowed Book (Never Return)" despite a simpler name being sufficient, requiring an explicit corrective prompt. These observations indicate that while ChatGPT generally captures the intended security semantics, it occasionally introduces unsupported inferences and over-elaborates on construct naming, both of which require modeler intervention to resolve.

RQ3: Visual Clarity and Notational Coherence

Visual presentation quality was the most consistently problematic dimension across all 10 case studies, and visual corrections accounted for most follow-up prompts issued. The most frequently recurring issue was construct label visibility. In two case studies, specifically the E-Voting and Healthcare systems, misuser labels were not visible in the generated output and proved resistant to correction across multiple follow-up prompts. Misuse case stereotype visibility against dark backgrounds was problematic in the Food Delivery and E-Voting systems. In the E-Voting case, the issue required three consecutive corrective prompts and was only resolved when a specific PlantUML styling parameter, `StereotypeFontColor`, was explicitly referenced, suggesting that high-level visual correction prompts are insufficient for resolving low-level rendering issues. In the Food Delivery system, ChatGPT applied an unsolicited transparent background to the diagram, producing an unintended checkerboard rendering pattern that required explicit correction. In the RFID system, the `<<include>>` stereotype label on a link between use cases was also not visible, indicating that stereotype visibility issues extend beyond misuse case stereotypes. ChatGPT also incorrectly applied dark background styling to a legitimate actor in the Library Management System, indicating that styling corrections can inadvertently affect unintended elements. These findings indicate

that while ChatGPT can generate notationally recognizable misuse case diagrams, it does not reliably produce diagrams with the visual clarity and cognitive coherence required for practical use without targeted visual prompting.

RQ4: Scalability

ChatGPT demonstrated no observable degradation in syntactic or semantic quality as case study complexity increased. The Swiss Banking system, the most complex case study with 24 security requirements and over 50 notational constructs, was completed successfully in only one prompt. In contrast, the Library Management System, one of the smallest case studies with 13 constructs, required four prompts. This inverse relationship between system size and prompt count suggests that model complexity does not negatively impact ChatGPT's ability to generate correct misuse case diagrams, and that prompt count is driven more by diagram-specific issues such as visual styling errors and relationship accuracy than by the overall scale of the modeling task. These findings provide encouraging evidence that ChatGPT can handle large and complex misuse case models without significant loss of correctness.

RQ5: Domain Sensitivity

ChatGPT did not exhibit sensitivity to the application domain in terms of semantic correctness. The model successfully generated semantically correct misuse case constructs across a wide range of domains including library management, transportation, cyber-physical systems, financial services, healthcare, and supply chain management. The visual and notational errors observed across the case studies did not appear to be domain-specific in nature and are more likely attributable to limitations in ChatGPT's handling of diagram styling and rendering rather than to gaps in domain knowledge. It is worth noting that all prompts in this study used descriptive and domain-specific construct names. Whether ChatGPT would generate semantically meaningful misuse case diagrams when given vague or abstract input labels remains an open question that warrants investigation in future work.

RQ6: Cross-Session Context Retention

The findings suggest that ChatGPT may retain contextual knowledge across chat sessions, representing a potential threat to the internal validity of the evaluation. This is evidenced by the pattern of prompt counts observed across trial runs, where the Library Management System, conducted first, required four prompts, while the subsequent RFID

and Ride Sharing sessions each required only two prompts despite being considerably more complex. One plausible explanation is that ChatGPT retained and reused knowledge from earlier sessions, resulting in fewer corrective prompts being required in later trials and potentially underestimating the true modeler effort that would be required in fully isolated sessions. It is therefore recommended that future evaluations of this nature are conducted with memory explicitly disabled to ensure that each trial run reflects the model's baseline capability independent of prior interactions.

5 CONCLUSIONS

This paper presented an exploratory empirical study evaluating the ability of ChatGPT to generate cybersecurity misuse case diagrams from natural language security requirements. Using 12 case studies of varying complexity spanning small, medium, and large systems across diverse application domains, we applied a structured iterative prompting protocol in which each trial run was evaluated against an expert-constructed ground-truth misuse case diagram across three dimensions: syntactic correctness, semantic correctness, and visual presentation quality.

The results demonstrate that ChatGPT can generate misuse case diagrams that achieve full alignment with ground-truth models across all 12 case studies, indicating strong overall performance. The model showed strength in semantic correctness and domain independence, performing consistently across diverse application domains and system sizes without observable degradation in quality. Visual presentation quality emerged as the most persistent challenge, requiring most corrective prompting effort across all trial runs. These findings suggest that ChatGPT holds genuine practical value as an AI-assisted modeling tool for security requirements engineering, particularly as a starting point for novice modelers, while expert oversight remains necessary to ensure visual and notational quality.

The findings of this study stand in notable contrast to those reported by Cámara et al. (2023), who evaluated ChatGPT-4 on general UML modeling tasks and concluded that its performance was limited, citing syntactic and semantic deficiencies, lack of consistency across responses, and significant scalability issues as complexity increased. In particular, Cámara et al. (2023) found semantic correctness to be the weakest dimension of ChatGPT-4's performance, observing that the model frequently produced models that were semantically misaligned

with user intent, and that domain sensitivity negatively influenced output quality. In contrast, our evaluation of ChatGPT-5 on the more specialized and security-specific task of misuse case diagram generation yielded markedly stronger results. All 12 trial runs were completed successfully, semantic correctness was consistently achieved across all case studies, and no degradation in quality was observed as system complexity scaled from small to large. Domain sensitivity was similarly absent, with ChatGPT-5 performing consistently across diverse application domains. These differences suggest that the advancements introduced in ChatGPT-5, particularly in reasoning depth and instruction-following precision, represent a meaningful leap in LLM modeling capability, and that conclusions drawn about LLM limitations from studies conducted with earlier model versions should be interpreted with caution in light of the rapid pace of model improvement.

Future work should investigate the impact of prompt formulation strategies on first-attempt diagram quality, with the aim of reducing the number of corrective prompts required. Evaluations conducted with memory explicitly disabled are also recommended to address the cross-session context retention concern identified in this study and ensure fully reproducible results. Additionally, extending the evaluation to a broader range of security modeling notations and to larger and more complex systems would further characterize the scalability boundaries of LLM-assisted misuse case modeling.

REFERENCES

- AbdulGhaffar, A., & Matrawy, A. (2025). LLMs' suitability for network security: A case study of STRIDE threat modeling (arXiv:2505.04101). arXiv. <https://doi.org/10.48550/arXiv.2505.04101>
- Abdelnabi, E. A., Maatuk, A. M., & Abdelaziz, T. M. (2021). An algorithmic approach for generating behavioral UML models using natural language processing. In *Proceedings of the 7th International Conference on Engineering and MIS 2021* (pp. 1–6).
- Alexander, I. (2003). Misuse cases: Use cases with hostile intent. *IEEE Software*, 20(1), 58–66.
- Aruna, S., & Varun, S. (2025). Automated diagram generation using GPT-4: An empirical evaluation of logical correctness.
- Borji, A. (2023). A categorical archive of ChatGPT failures. *arXiv preprint arXiv:2302.03494*.
- Cámara, J., Troya, J., Burgueño, L., & Vallecillo, A. (2023). On the assessment of generative AI in modeling tasks: An experience report with ChatGPT and UML. *Software and Systems Modeling*, 22(3), 781–793.

- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Edwards, H., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- El-Attar, M. (2012a). A framework for improving quality in misuse case models. *Business Process Management Journal*, 18(2), 168–196.
- El-Attar, M. (2012b). Towards developing consistent misuse case models. *Journal of Systems and Software*, 85(2), 323–339.
- El-Attar, M. (2014a). Using SMCD to reduce inconsistencies in misuse case models: A subject-based empirical evaluation. *Journal of Systems and Software*, 87, 104–118.
- El-Attar, M. (2014b). From misuse cases to mal-activity diagrams: Bridging the gap between functional security analysis and design. *Software and Systems Modeling*, 13(1), 173–190.
- El-Attar, M., Luqman, H., Karpati, P., Sindre, G., & Opdahl, A. L. (2015). Extending the UML statecharts notation to model security aspects. *IEEE Transactions on Software Engineering*, 41(7), 661–690.
- Firesmith, D. (2003). Security use cases. *Journal of Object Technology*, 2(3), 53–64.
- France, R., & Rumpe, B. (2007). Model-driven development of complex software: A research roadmap. In *Proceedings of the 2007 Future of Software Engineering* (pp. 37–54). IEEE Computer Society.
- Howard, M., & Lipner, S. (2006). *The security development lifecycle*. Microsoft Press.
- Jacobson, I. (1992). *Object-oriented software engineering: A use case driven approach*. Addison-Wesley.
- Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2024). A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*.
- Kamalrudin, M. (2013). Automated support for managing security requirements. In *Proceedings of the 35th International Conference on Software Engineering* (pp. 1–4).
- Khan, Y., & El-Attar, M. (2026). *LLM-based modeling: Misuse cases* [Data set, supplementary materials and trial results]. GitHub. <https://github.com/ybakhan/llm-based-modeling/tree/main/misusecases> [Last Accessed: March 6th, 2026]
- Khojah, R., Mohamad, M., Leitner, P., & Gorschek, T. (2024). Beyond code generation: An observational study of ChatGPT usage in software engineering practice. In *Proceedings of the 46th International Conference on Software Engineering* (pp. 1–13).
- Mbaka, J., & Tuma, P. (2024). Evaluating STRIDE threat identification using LLM-generated data flow diagrams.
- Mollaefar, M., Bissoli, A., Van Landuyt, D., & Ranise, S. (2025). PILLAR: LINDDUN privacy threat modeling using LLMs. In *2025 IEEE European Symposium on Security and Privacy Workshops (EuroSandPW)* (pp. 278–286).
- Nguyen, N., & Nadi, S. (2022). An empirical evaluation of GitHub Copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories* (pp. 1–5).
- OWASP Foundation. (2021). *OWASP threat modeling process*. <https://owasp.org>
- Saha, T., Bhatt, D., Kalle, E., & Saha, S. (2025). ThreatLens: LLM-guided threat analysis and test planning for hardware security. *arXiv preprint*.
- Saleh, F., & El-Attar, M. (2015). A scientific evaluation of the misuse case diagrams visual syntax. *Information and Software Technology*, 66, 73–96.
- Shostack, A. (2014). *Threat modeling: Designing for security*. John Wiley & Sons.
- Sindre, G., & Opdahl, A. L. (2001). Capturing security requirements through misuse cases. In *Proceedings of the Norwegian Informatics Conference* (pp. 219–230).
- Sindre, G., & Opdahl, A. L. (2005). Eliciting security requirements with misuse cases. *Requirements Engineering*, 10(1), 34–44.
- Sridhara, G., & Mazumdar, S. (2023). ChatGPT: A study on its utility for ubiquitous software engineering tasks. *arXiv preprint arXiv:2305.16837*.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer.
- Wu, T., Yang, S., Liu, S., Nguyen, D., Jang, S., & Abuadba, A. (2024). ThreatModeling-LLM: Automating threat modeling using large language models for banking system. *arXiv preprint arXiv:2411.17058*.
- Yang, S., & Sahraoui, H. (2022). Towards automatically extracting UML class diagrams from natural language specifications. In *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings* (pp. 396–403).